



| The European Synchrotron

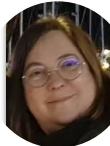
A dive into mxcube



Collaborating since 2005



Beamline control and software development



Antonia Beteva



Yan Walesch



Mathis Loussert
(Alternance)

Scientific coordination



Daniele De Sanctis

We never really use mxcube core on its own but you can, for fun (or for testing)

Checkout repository

<https://github.com/mxcube/mxcubecore.git>

Instructions: <https://github.com/mxcube/mxcubeweb>

Setup your environment

**(conda-environment-dev.yml contains the development libraries/tools needed)
poetry or pip install**

Run the tests

Simply running pytest

What is mxcubecore - A control system agnostic library for sharing common routines and integration of instrumentation

These objects, routines and instrumentation interface logic, is implemented as HardwareObjects

Let's have a look at some code:

Simple Example

```
import os
from gevent import monkey
monkey.patch_all(thread=False)

import mxcore
from mxcore import HardwareRepository as HWR

ROOT_DIR = os.path.abspath(os.path.dirname(mxcore.__file__))

hwr_config_path = "%s%s%s" % (
    os.path.join(ROOT_DIR, "configuration/mockup"),
    ":",
    os.path.join(ROOT_DIR, "configuration/mockup/test"),
)

print("Configuration path:")
print(hwr_config_path)

HWR.init_hardware_repository(hwr_config_path)

def print_value(val):
    print(val)

HWR.beamline.resolution.connect("valueChanged", print_value)
HWR.beamline.resolution.get_value()
HWR.beamline.resolution.set_value(1.4)
```

- **Gevent**
- **HardwareRepository** is accessed as **HWR**

Simple example - output

```
DEBUG.HWR.DiffractometerMockup.moving motor {motor} to position {position}
```

role	Class	file	Time (ms)	Comment
beamline	Beamline	beamline_config.yml	100	Start loading contents:
machine_info	MachineInfoMockup	mach-info-mockup.xml	37	
transmission	TransmissionMockup	transmission-mockup.xml	2	
energy	EnergyMockup	energy-mockup.xml	1	
beam	BeamMockup	beam-mockup.xml	4	
flux	FluxMockup	flux-mockup.xml	268	
detector	DetectorMockup	detector-mockup.xml	5	
resolution	ResolutionMockup	resolution-mockup.xml	1	
safety_shutter	ShutterMockup	safety-shutter-mockup.xml	2	
sample_changer	SampleChangerMockup	sample-changer-mockup.xml	9	
diffractometer	DiffractometerMockup	diffractometer-mockup.xml	738	
sample_view	SampleView	sample-view-mockup.xml	2	
mock_procedure	ProcedureMockup	procedure-mockup.yml	103	
beamline	Beamline	beamline_config.yml	1278	Done loading contents

The beamline object, `HWR.beamline`, is used to access the various configured hardware objects

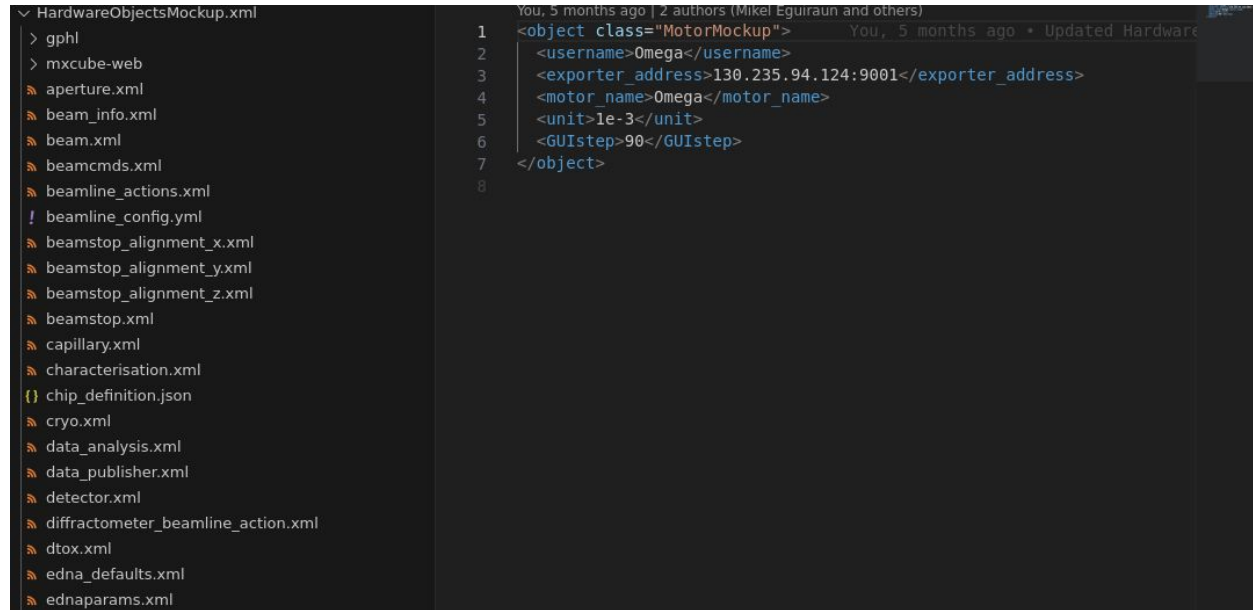
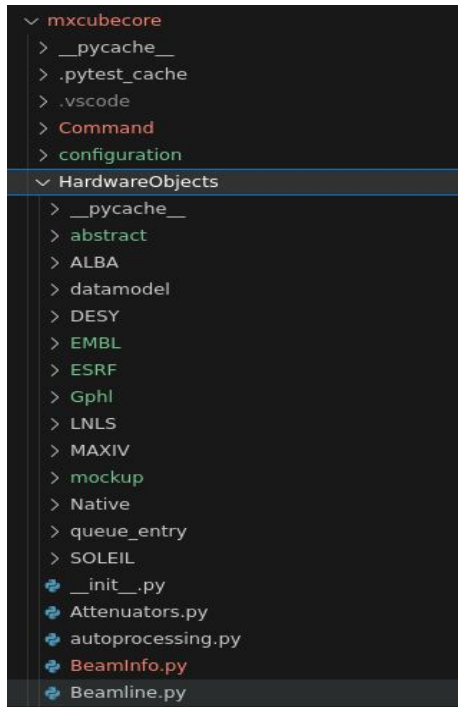
But how does this work (in a nutshell :)) ?

```
DEBUG:HWK.DiffractometerMockup.moving motor {motor} to position {position}
```

role	Class	file	Time (ms)	Comment
beamline	Beamline	beamline_config.yml	100	Start loading contents:
machine_info	MachineInfoMockup	mach-info-mockup.xml	37	
transmission	TransmissionMockup	transmission-mockup.xml	2	
energy	EnergyMockup	energy-mockup.xml	1	
beam	BeamMockup	beam-mockup.xml	4	
flux	FluxMockup	flux-mockup.xml	268	
detector	DetectorMockup	detector-mockup.xml	5	
resolution	ResolutionMockup	resolution-mockup.xml	1	
safety_shutter	ShutterMockup	safety-shutter-mockup.xml	2	
sample_changer	SampleChangerMockup	sample-changer-mockup.xml	9	
diffractometer	DiffractometerMockup	diffractometer-mockup.xml	738	
sample_view	SampleView	sample-view-mockup.xml	2	
mock_procedure	ProcedureMockup	procedure-mockup.yml	103	
beamline	Beamline	beamline_config.yml	1278	Done loading contents

- The configuration files are parsed and the hardware objects loaded
- Starting with the “beamline object” and traversing the “hierarchy” downwards (order in configuration file) loading the children
 - yaml files loaded via `HardwareRepository.load_from_yaml`
 - xml files loaded via `HardwareRepositoryClient._load_hardware_object`
- Generation of YAML from XML is possible `--export-yaml-config` - can be used for migration

HardwareObjects in action



- Each site has its own directory for site specific HardwareObjects and code
- As you now know there is a folder with yaml and xml file configuring these objects
- We are slowly moving away from XML configuration files

HardwareObjects in action

```
<?xml version="1.0" encoding="UTF-8"?>
<object class = "DetectorMockup">
  <username>detector</username>
  <object hwrid="/dtox" role="detector_distance"/>
  <tempThreshold>33.5</tempThreshold>
  <humidityThreshold>20.0</humidityThreshold>
  <tolerance>0.2</tolerance>
  <type>Eiger2</type>
  <model>9M</model>
  <manufacturer>DECTRIS</manufacturer>
  <px>0.075</px>
  <py>0.075</py>
  <width>3110</width>
  <height>3269</height>
  <hasShutterless>True</hasShutterless>
  <fileSuffix>cbf</fileSuffix>
  <roi_mode_list>["0", "C2", "C16"]</roi_mode_list>
  <beam>
    <!-- Values are in pixels/mm and pixels resp. -->
    <ax>0.0</ax>
    <ay>0.0402</ay>
    <bx>1565.715</bx>
    <by>1702.058</by>
  </beam>
  <exports>["restart"]</exports>
</object>
```

```
class DetectorMockup(AbstractDetector):
    """
    Descript. : Detector class. Contains all information about detector
               the states are 'OK', and 'BAD'
               the status is busy, exposing, ready, etc.
               the physical property is RH for pilatus, P for rayonix
    """

    def __init__(self, name):
        """
        Descript. :
        """
        AbstractDetector.__init__(self, name)

    def init(self):
        """
        Descript. :
        """
        AbstractDetector.init(self)

        self.temperature = 25
        self.humidity = 60
        self.actual_frame_rate = 50
        self.roi_modes_list = ast.literal_eval(
            self.get_property("roi_mode_list", '["0", "C2", "C16"]')
        )
        self.roi_mode = 0
        self.exposure_time_limits = eval(
            self.get_property("exposure_time_limits", "[0.04, 60000]")
        )
```

- **Important:** class name and file name should be the same, also note that the YAML class name contains the python module name
- XML is **not validated** against any schema, we often use **ast.literal_eval** to evaluate python structures.
- YAML files **will be validated** against a PyDantic model

```
1 #YAML 1.2
2 ---
3 class: DetectorMockup.DetectorMockup
4 configuration:
5   beam:
6     ax: 0.0
7     ay: 0.0402
8     bx: 1565.715
9     by: 1702.058
10   exports: '["restart"]'
11   fileSuffix: cbf
12   hasShutterless: true
13   height: 3269
14   humidityThreshold: 20.0
15   manufacturer: DECTRIS
16   model: 9M
17   px: 0.075
18   py: 0.075
19   tempThreshold: 33.5
20   tolerance: 0.2
21   type: Eiger2
22   username: detector
23   width: 3110
24   roi_mode_list: '["0", "C18", "C12", "C2"]'
25 objects:
26   detector_distance: dtox.yaml
```

HardwareObjects in action

```
detectorMockup.py  detector.xml  DetectorMockup.py  DetectorMockup.py
You, 11 seconds ago | 3 authors (Marcus Oskarsson and others)
<object class = "DetectorMockup">
  <username>detector</username>
  <object hwrid="/dtox" role="detector_distance"/>
  <tempThreshold>33.5</tempThreshold>
  <humidityThreshold>20.0</humidityThreshold>
  <tolerance>0.2</tolerance>
  <type>Eiger2</type>
  <model>9M</model>
  <manufacturer>DECTRIS</manufacturer>
  <px>0.075</px>
  <py>0.075</py>
  <width>3110</width>
  <height>3269</height>
  <hasShutterless>True</hasShutterless>
  <fileSuffix>cbf</fileSuffix>
  <roi_mode_list>["0", "C2", "C16"]</roi_mode_list>
  <beam>
    <!-- Values are in pixels/mm and pixels resp. -->
    <ax>0.0</ax>
    <ay>0.0402</ay>
    <bx>1565.715</bx>
    <by>1702.058</by>
  </beam>
  <exports>["restart"]</exports>
</object>
```

```
class DetectorMockup(AbstractDetector):
    """
    Descript. : Detector class. Contains all information about detector
               the states are 'OK', and 'BAD'
               the status is busy, exposing, ready, etc.
               the physical property is RH for pilatus, P for rayonix
    """
    def __init__(self, name):
        """
        Descript. :
        """
        AbstractDetector.__init__(self, name)

    def init(self):
        """
        Descript. :
        """
        AbstractDetector.init(self)

        self.temperature = 25
        self.humidity = 60
        self.actual_frame_rate = 50
        self.roi_mode_list = ast.literal_eval(
            self.get_property("roi_mode_list", '["0", "C2", "C16"]')
        )
        self.roi_mode = 0
        self.exposure_time_limits = eval(
            self.get_property("exposure_time_limits", "[0.04, 60000]")
        )
```

- A configured property values in `<start_tag></end_tag>` or in the YAML configuration section are retrieved with `get_property("tag")` or via the `.config` member
- objects defined with `<object />` or in the objects section are retrieved by `get_object_by_role("role")` using the role name

```
1  %YAML 1.2
2  ---
3  class: DetectorMockup.DetectorMockup
4  configuration:
5    beam:
6      ax: 0.0
7      ay: 0.0402
8      bx: 1565.715
9      by: 1702.058
10   exports: ['restart']
11   fileSuffix: cbf
12   hasShutterless: true
13   height: 3269
14   humidityThreshold: 20.0
15   manufacturer: DECTRIS
16   model: 9M
17   px: 0.075
18   py: 0.075
19   tempThreshold: 33.5
20   tolerance: 0.2
21   type: Eiger2
22   username: detector
23   width: 3110
24   roi_mode_list: ["0", "C10", "C12", "C2"]
25   objects:
26     detector_distance: dtox.yaml
```

Control system integration

HardwareObjects can be integrated with control systems via something called channels and commands

Direct implementation via a proxy object, i.e beamline.control - a more recent approach

Antonia will talk more about this :)

HardwareObjects can communicate with control systems via something called channels and commands

```
class Microdiff(MiniDiff.MiniDiff):
    def init(self):
        self.phiMotor = self.get_object_by_role("phi")
        self.exporter_addr = self.get_property("exporter_address")

        self.update_state([HardwareObjectState.READY])

        self.x_calib = self.add_channel(
            {
                "type": "exporter",
                "exporter_address": self.exporter_addr,
                "name": "x_calib",
            },
            "CoaxCamScaleX",
        )
        self.y_calib = self.add_channel(
            {
                "type": "exporter",
                "exporter_address": self.exporter_addr,
                "name": "y_calib",
            },
            "CoaxCamScaleY",
        )
```

Channels and commands provide an abstraction for the various control systems used

Channels and commands provide an abstraction for the various control systems used

Supported are: Exporter (EMBL), Tango, Taco, Tine, Sardana, EPICS, (SPEC), *BLISS (*Not via channels and commands*)

- *Channels for values (and events)*
- *Commands for “functions” methods executed on the other end*

HardwareObjects inherit **CommandContainer** meaning that we can use **add_channel** and **add_command**

- The library **pyDispatcher** is used for handling signals/events
- Each protocol implements a **CommandObject**, a **ChannelObject** and protocol specific “Client” for handling events and other protocol internals.
- A channel emits a **“update”** signal with the new value on a event

Antonia will talk more about this (if you like) ?

- The mxcube core work have harmonised the API we use for these **HardwareObjects**: (summary of changes: <https://github.com/mxcube/mxcubecore/blob/develop/changelog.txt>)
- *There are still some work to be done for instance with signals and further refining the interface of certain objects*
- A base class for all Hardware Objects **HardwareObject** and has a well set of state; READY, BUSY, OFF, FAULT etc.
- A set of commonly used subclasses derived from HardwareObject;
- Introduction of **BeamlineObject** for facilitating access to “well known” HardwareObjects

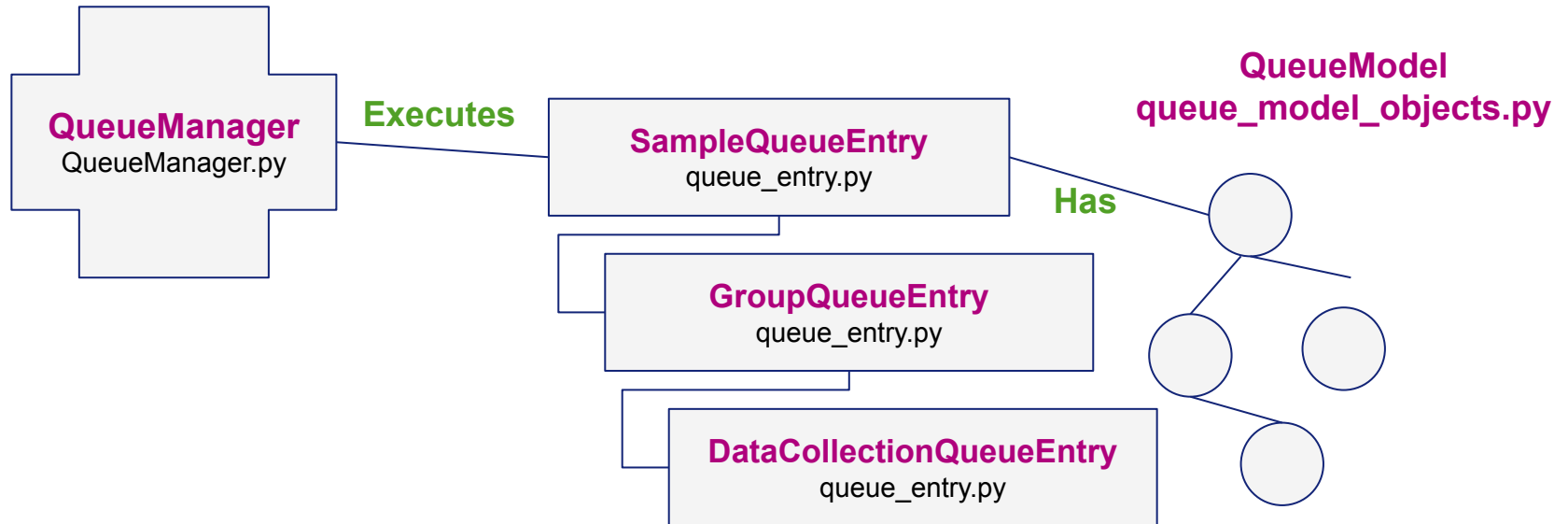
- The basic objects we use are, `HardwareObject`, `AbstractActuator`,
- From these there are a number of objects derived that everybody most likely will use or can reuse: `AbstractNState`, `AbstractMotor`, `AbstractTransmission`, `AbstractResolution`, `AbstractDetector`, `AbstractShutter`, `AbstractSlits`, `AbstractBeam` and ... actually the list is quite long :)
- To keep in mind:
 - `HardwareObject` provides `set_state` and `get_state`
 - `AbstractActuator` provides `set_value`, `get_value` and `set_limits`, and `get_limits`
 - `AbstractNState` is a `AbstractActuator` where the value is in finite set of values (states)
- Thanks to Antoina:
<https://github.com/mxcube/mxcubecore/blob/develop/Hierarchy.pdf>

About the queue

(Ok, take a deep breath ;))

- HardwareObjects are important for instrument control
- Data acquisition tasks/protocols are implemented as QueueEntry objects (not a HardwareObject)
- These QueueEntryObjects are executed via a queue (QueueManager)
- The data used by the collection routine is defined in a QueueModel
- The queue is executed by the QueueManager, the logic is defined in a QueueEntryObject and the data is contained in the QueueModel
- The queue further provides means for automation, via XMLRPC

<https://mxcube.core.readthedocs.io/en/stable/dev/queue.html>

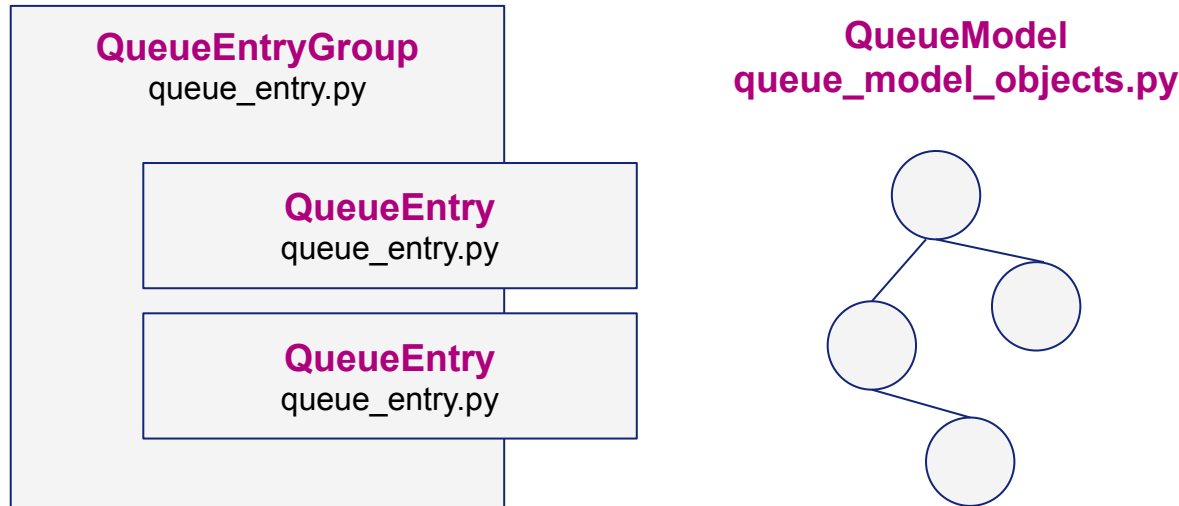


- **QueueManager** executes **QueueEntry** objects each having data models.
- Each **QueueEntry** in turn is a tree with Parent and child nodes, where the tree is traversed downwards (depth first), children gets executed after the parent
- Each entry has a **pre and post execute** that gets executed **before and main execute function**
- In **practice maximum three levels of QueueEntry** objects are used, what's called **SampleQueueEntry, GroupQueueEntry** and **QueueEntry**. (Workflows sometimes uses more)

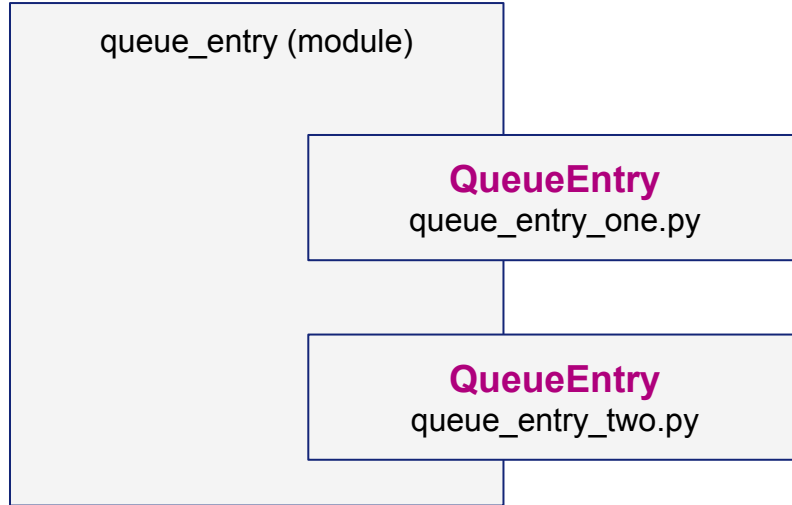
Defining a new collection sequence

Two approaches:

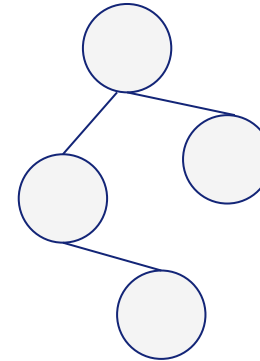
- The original static approach where the queue entry and models are explicitly coded into the application
- The more recent approach where the queue entry is loaded via configuration, the model is defined as a pydantic model and the UI generated based on the model
- Both approaches uses the same queue system and bases classes; **QueueManager**, **QueueEntry** and **QueueModel** - *The difference is how they are expressed and loaded*



- QueueModels are written as python objects that gets converted to dicts when passed around to **AbstractMutliCollect** or **AbstractCollect**, hard to debug and know what to pass.
- Creation of each type of Queue entry is more or less manually wired for creation in the UI
- **AbstractMutliCollect** and **AbstractCollect** implements common logic for a collection, the approach is not in itself bad, but the encapsulation could be better and we only need **ONE** abstract collect

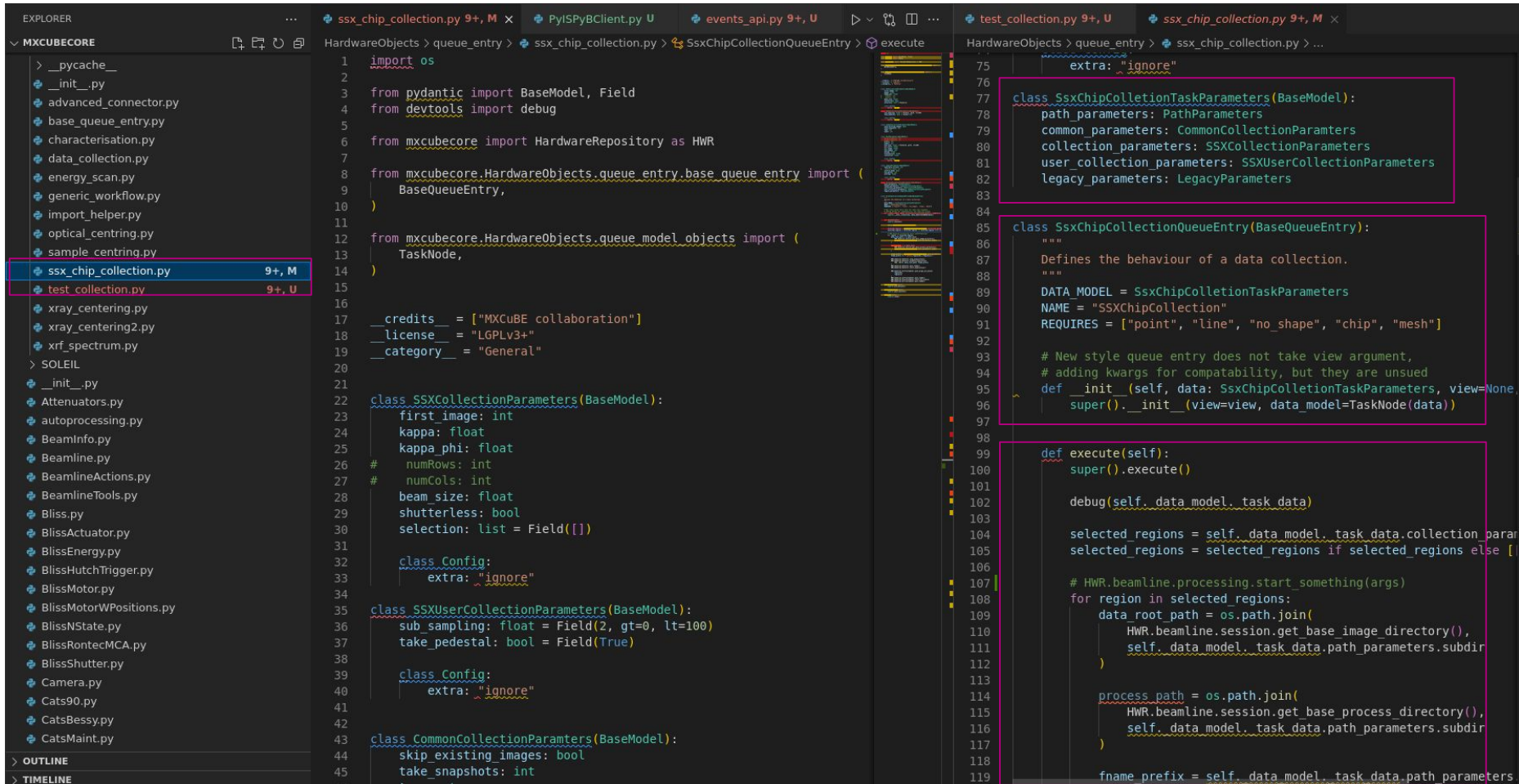


CommonQueueModels



- Each QueueEntry is self **contained in its own file with its specific models**
- Models are expressed as **Pydantic models**
- All common models are **shared in common queue models module**
- Each QueueEntry object in the queue_entry module (or other search path) is **dynamically imported**
- **A generic UI interface/dialog** can be built based on the **model data associated with a QueueEntry** (an evolution of the current workflow dialog)
- Common collect logic is shared in a BaseQueueEntry, moving away from *AbstractCollect

Example - New style queue entry



The screenshot displays a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with a folder named 'MXCUBECORE' containing various Python files. The code editor shows the implementation of a new style queue entry in 'ssx_chip_collection.py'.

```
1 import os
2
3 from pydantic import BaseModel, Field
4 from devtools import debug
5
6 from mxcubecore import HardwareRepository as HWR
7
8 from mxcubecore.HardwareObjects.queue_entry.base_queue_entry import (
9     BaseQueueEntry,
10 )
11
12 from mxcubecore.HardwareObjects.queue_model.objects import (
13     TaskNode,
14 )
15
16
17 _credits_ = ["MXCuBE collaboration"]
18 _license_ = "LGPLv3+"
19 _category_ = "General"
20
21
22 class SSXCollectionParameters(BaseModel):
23     first_image: int
24     kappa: float
25     kappa_phi: float
26     # numRows: int
27     # numCols: int
28     beam_size: float
29     shutterless: bool
30     selection: list = Field([])
31
32     class Config:
33         extra: "ignore"
34
35 class SSXUserCollectionParameters(BaseModel):
36     sub_sampling: float = Field(2, gt=0, lt=100)
37     take_pedestal: bool = Field(True)
38
39     class Config:
40         extra: "ignore"
41
42 class CommonCollectionParameters(BaseModel):
43     skip_existing_images: bool
44     take_snapshots: int
```

The code editor also shows the implementation of the 'SsxChipCollectionTaskParameters' and 'SsxChipCollectionQueueEntry' classes, which define the behavior of a data collection. The 'SsxChipCollectionTaskParameters' class is a Pydantic model that inherits from 'BaseModel' and defines parameters for a data collection. The 'SsxChipCollectionQueueEntry' class inherits from 'BaseQueueEntry' and defines the behavior of a data collection, including the 'execute' method.

```
75 extra: "ignore"
76
77 class SsxChipCollectionTaskParameters(BaseModel):
78     path_parameters: PathParameters
79     common_parameters: CommonCollectionParameters
80     collection_parameters: SSXCollectionParameters
81     user_collection_parameters: SSXUserCollectionParameters
82     legacy_parameters: LegacyParameters
83
84
85 class SsxChipCollectionQueueEntry(BaseQueueEntry):
86     """
87     Defines the behaviour of a data collection.
88     """
89     DATA_MODEL = SsxChipCollectionTaskParameters
90     NAME = "SSXChipCollection"
91     REQUIRES = ["point", "line", "no_shape", "chip", "mesh"]
92
93     # New style queue entry does not take view argument,
94     # adding kwargs for compatability, but they are unused
95     def __init__(self, data: SsxChipCollectionTaskParameters, view=None):
96         super().__init__(view=view, data_model=TaskNode(data))
97
98
99     def execute(self):
100         super().execute()
101
102         debug(self.data_model.task_data)
103
104         selected_regions = self.data_model.task_data.collection_parameters.selected_regions
105         selected_regions = selected_regions if selected_regions else []
106
107         # HWR.beamline.processing.start something(args)
108         for region in selected_regions:
109             data_root_path = os.path.join(
110                 HWR.beamline.session.get_base_image_directory(),
111                 self.data_model.task_data.path_parameters.subdir
112             )
113
114             process_path = os.path.join(
115                 HWR.beamline.session.get_base_process_directory(),
116                 self.data_model.task_data.path_parameters.subdir
117             )
118
119             fname_prefix = self.data_model.task_data.path_parameters
```

Example UI

MXCuBE-Web (SSX-CHIP)

Samples Data collection Equipment System log

Energy: 12.4000 KeV Resolution: ~ Å Transmission: 10.0 %
Wavelength: 1.00 Å Detector: 10.1 mm Flux: 1.50e+10 ph/s

Detector: UNKNOWN Sample Changer: READY Capillary: UNKNOWN Fast Shutter: CLOSED Safety shutter: CLOSED

Beamline Actions ▾

Beam size: 50 ▾

Chip (Diamond Chip):
Navigate

Omega: 359.90 90.0°

X: 0.000 0.1 mm

Y: -49.900 0.1 mm

Sample alignment:
Show motors ▾

Snapshot

Path: /data/id29/inhouse/opid291/20220916/RAW_DATA/Mb/Mb-Mb/
Filename: Mb-Mb_[RUN#]_[IMG#]
Subdirectory: Mb/Mb-Mb/
Prefix: Mb-Mb

Acquisition

Exp Time: 0.02 Align Chip ☒
Sub Sampling: 4 Take Pedestal ☒

Default Parameters Run Now Add to Queue

Sample: Mb - Mb Queued Samples (0)

SSXFoilCollection
SSXFoilCollection
SSXInjectorCollection
SSXInjectorCollection
SSXInjectorCollection
SSXInjectorCollection

Log messages:
[10:15:35]: Opening OH2 safety shutter
[10:15:35]: Not mounting next sample automatically (Auto mount next)
[10:15:29]: Acquired 1000 images

50 μm

Defining a new collection sequence

We would recommend to use the dynamically imported queue entries

We might still need to refine and improve some bits - but we are glad to do so with your input.

Thank you for your attention

